

Sequence-to-Sequence (Seq2Seq) Models

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

Week 7 Contents

- Sequence Learning Problems
- Introduction to Recurrent Neural Networks
 - Architecture of RNN
 - Types of RNN
- Numerical Example: Forward Computation in RNN (Step-by-step)
- Backpropagation Through Time (BTT)
- Challenges such as Vanishing and Exploding Gradients
- Hands-on: Implementation of RNN using Keras (Time-series example)

Week 8: We will explore [LSTM](#) and [GRU](#) with hands-on

Today's Session Overview

Sequence = [29°C, 30°C, 31°C, 30°C, 28°C, 27°C]

Tomorrow's temperature depends on the sequence of past temperature values. ✓

Words typed = ["Deep", "Learning", "is", "a"]

Google predicts the next word based on this sequence.

- We will understand why ANN/CNN structures fail for sequential data.
- We will learn the **RNN architecture**. ✓
- Learn how RNNs remember previous inputs while processing the next ones using **recurrent connections**. ✓
- Discuss the **types of RNNs**.

How Feedforward Networks Process Data

Feedforward NN:

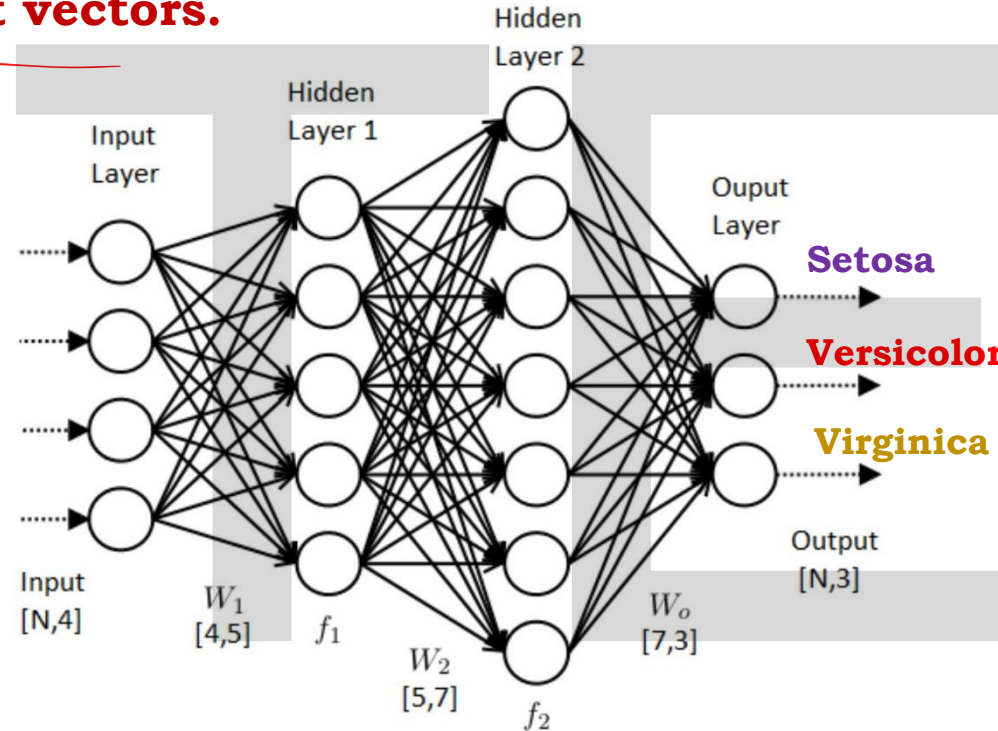
- Processes each row independently.
- Does *not remember* what it saw before.
- Does *not care* about the order of samples: It does not matter if row 1 comes before row 2 or after.

There is no **temporal or sequential relationship between the rows of the dataset.**

Feedforward NN, expects **fixed-length input vectors.**

	Sepal Length	Sepal Width	Petal Length	Petal Width	Class	
Xtrain	5.1	3.5	1.4	0.2	setosa	Y-Train
	4.9	3	1.4	0.2	setosa	
	7	3.2	4.7	1.4	versicolor	
	6.4	3.2	4.5	1.5	versicolor	
	6.9	3.1	4.9	1.5	versicolor	
	6.6	3	4.4	1.4	versicolor	
	4.7	3.2	1.3	0.2	setosa	
	5.1	3.5	1.4	0.3	setosa	
	5.2	3.5	1.5	0.2	setosa	
	6.3	2.9	5.6	1.8	virginica	
	6.5	3	5.8	2.2	virginica	
	7.6	3	6.6	2.1	virginica	
	5.2	3.4	1.4	0.2	setosa	
	4.7	3.2	1.6	0.2	setosa	
	4.8	3.1	1.6	0.2	setosa	
	5.4	3.4	1.5	0.4	setosa	
	6.8	2.8	4.8	1.4	versicolor	

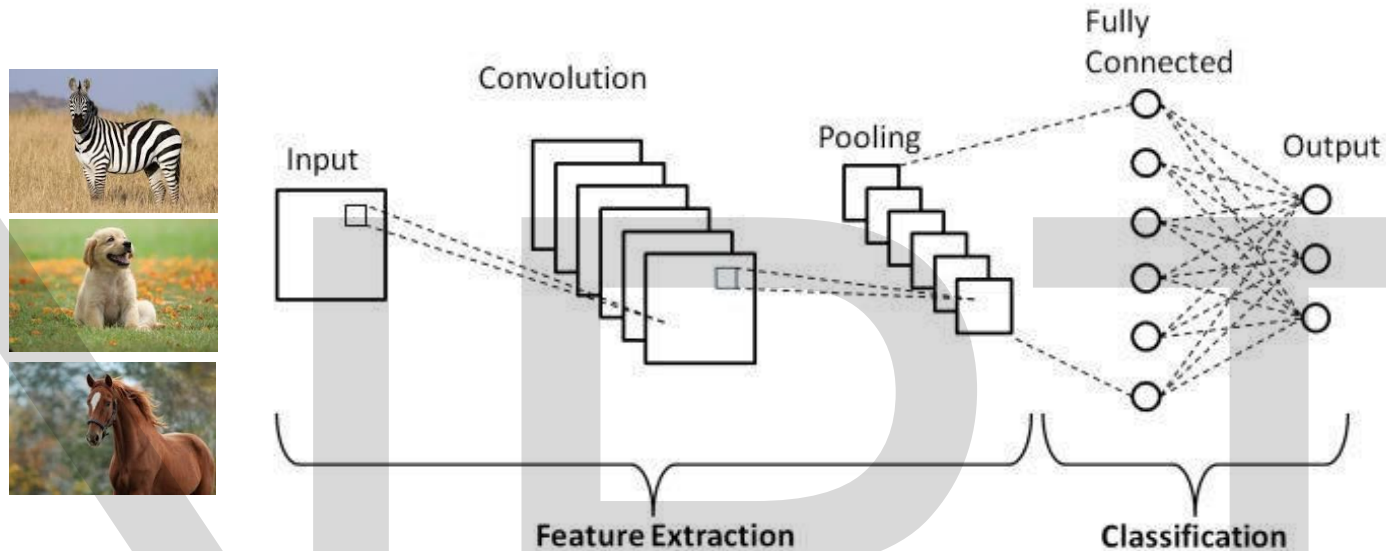
XTest	6.7	3	5	1.7
XTest	7.1	3	5.9	2.1
XTest	6.7	3	5	1.7
	6	2.9	4.5	1.5
	5.8	2.7	5.1	1.9
	7.1	3	5.9	2.1
	4.9	2.5	4.5	1.7
	7.3	2.9	6.3	1.8
	6.7	2.5	5.8	1.8



Each input sample is treated independently (Training samples are learned independently, and testing predictions depend only on the current input—never on previous inputs) — **the model does not remember previous samples.**

In CNN, the size of the input is always fixed

Images are resized to a fixed shape (e.g., 64×64 or 128×128) before passing to the model.



CNN also treats each input independently. Each image is processed **independently** of all others. Here also during training, images are learned independently, and during testing, predictions depend only on the current image—never on previous inputs.

Problem with Feedforward NN and CNN

- Model reads each of these inputs independently.
- There is no dependence between the inputs and the size of the input is fixed.
- There is no temporal or sequential relationship between inputs.

This is how ANN and CNN processes the input data and the working mechanism

Consider these examples:

Day	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri	Mon
Items sold	9	7	6	10	8	7	11	9	6	12	?

To predict Monday's sales, shop owner must look at the previous pattern (trend). So, the prediction depends on a **sequence** of past values. The inputs are dependent on each other.

Z				
Z	E			
Z	E	B		
Ignored	E	?		

Text Example:

To predict the next character, the model must remember the previous input.

Input is not independent and the order of input matters.

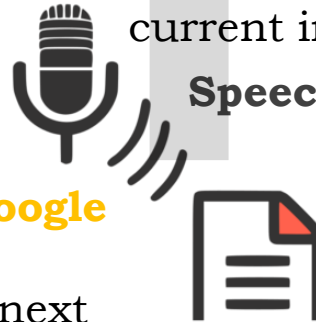


Video/Action Recognition:

punching

A single frame has no meaning. The action is understood from a sequence of frames. Order is important and the current input depends on the previous.

Speech to Text



Natural Language Example (Google Search Autocomplete):

The model cannot generate the next word without using the earlier words in the sentence. This again shows that there is a dependence and the order of words matters.

Google Search

I'm Feeling Lucky

Sequence Learning Problems

- The order of inputs matters.
- The current input depends on previous inputs.
- The input length is not fixed.

This is why ANN/CNN structures fail

Time-Series Example (Sales record of a shop): Yesterday's sales affect today's sales.

Stock market prediction, Weather forecasting

We need to build the model for such applications where:

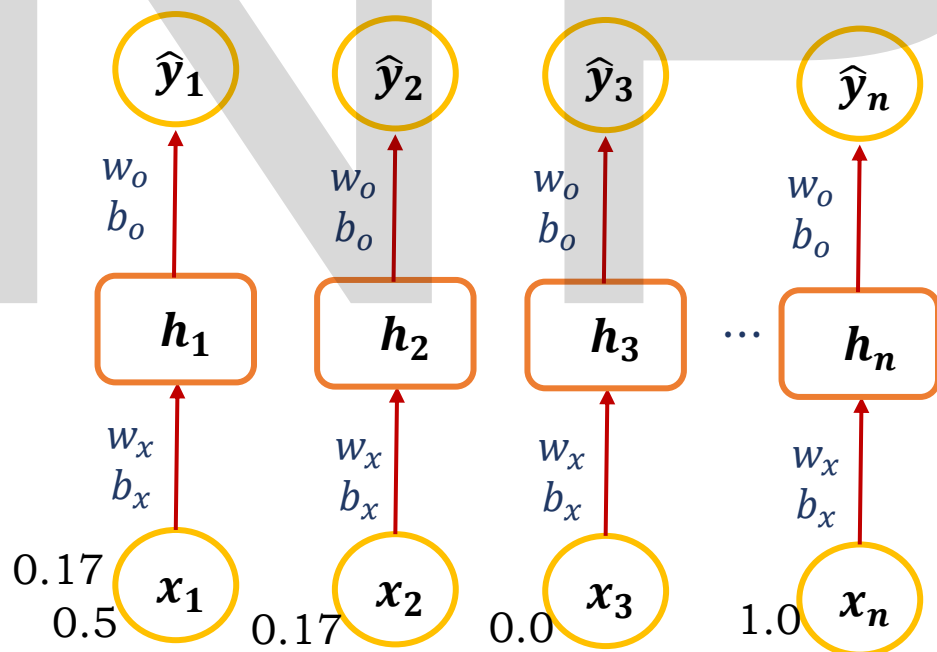
1. The model must **handle dependencies between inputs** ✓
2. The model must **handle variable-length inputs**

To achieve (1) and (2), At each time step, **the function executed by the model should be the same.**

	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
	9	7	6	10	8	7	11	9	6	12
X	0.5	0.17	0.0	0.67	0.33	0.17	0.83	0.5	0.0	1.0
Y	0.17	0.0	0.67	0.33	0.17	0.83	0.5	0.0	1.0	

The sales data across several days.
Predict the next value in a time series.

$$x_{norm} = \frac{x - \min(x)}{\max(x) - \min(x)}$$



$$h_1 = \tanh(x_1 w_x + b_x)$$

$$\hat{y}_1 = f(h_1 w_o + b_o)$$

$$h_2 = \tanh(x_2 w_x + b_x)$$

$$\hat{y}_2 = f(h_2 w_o + b_o)$$

$$h_i = \tanh(x_i w_x + b_x)$$

$$\hat{y}_i = f(h_i w_o + b_o)$$

We need to build the model for such applications where:

1. The model must **handle dependencies between inputs**
 2. The model must **handle variable-length inputs**
- To achieve (1) and (2), At each time step, **the function executed by the model should be the same.**

Recurrent connection in the network

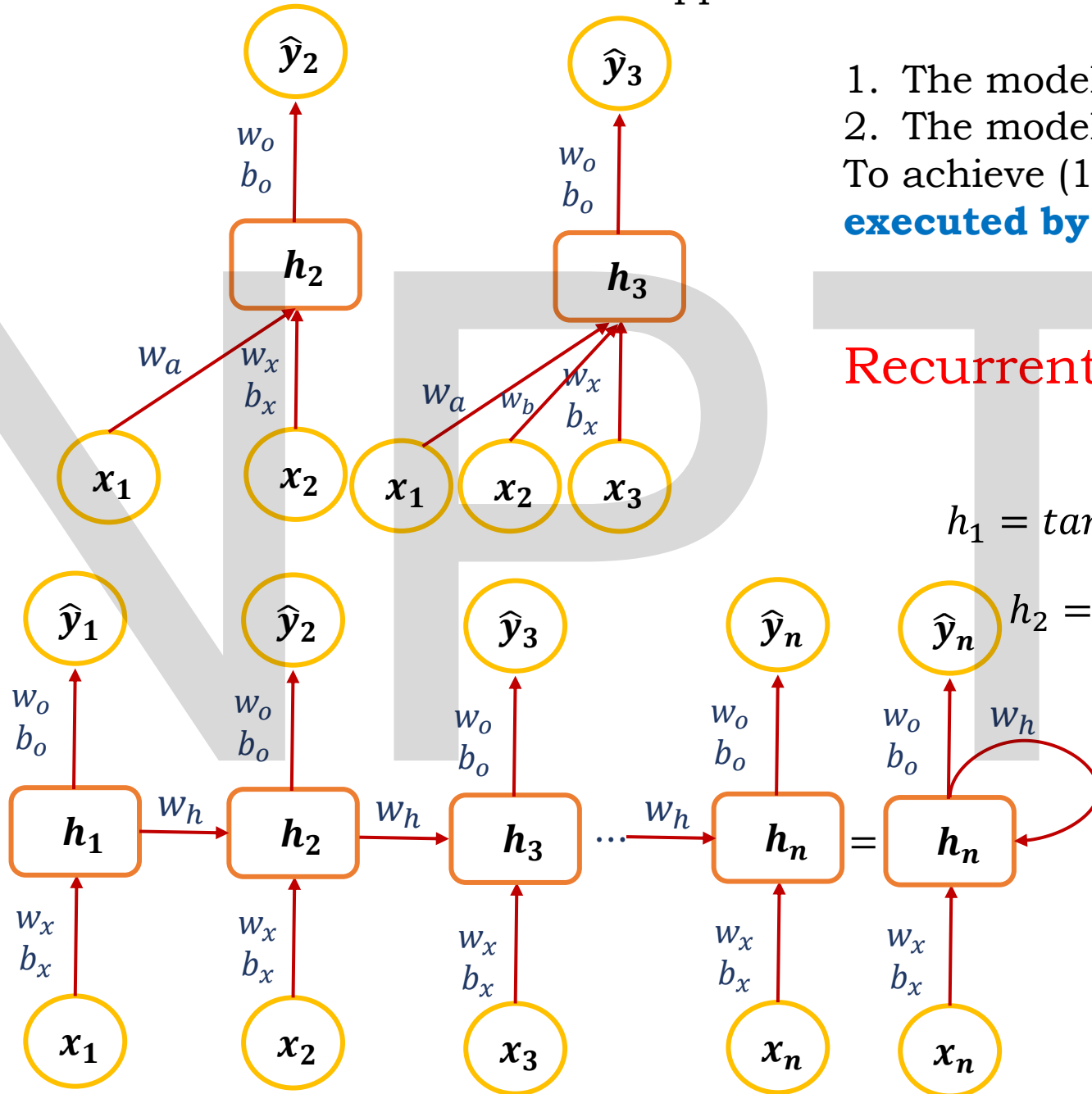
$$h_1 = \tanh(x_1 w_x + h_0 w_h + b_x) \quad \hat{y}_1 = f(h_1 w_o + b_o)$$

$$h_2 = \tanh(x_2 w_x + h_1 w_h + b_x) \quad \hat{y}_2 = f(h_2 w_o + b_o)$$

$$h_i = \tanh(x_i w_x + h_{i-1} w_h + b_x)$$

$$\hat{y}_i = f(h_i w_o + b_o)$$

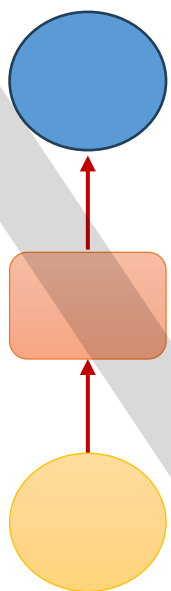
RNN architecture



RNN: Types

Sunset

Output: Prediction at time t+1



One to One

- ✓ Equivalent to FFNN with single input and single output.
- ✓ This configuration serves as a building block for understanding more complex architectures.

x_1 : Temperature at time t



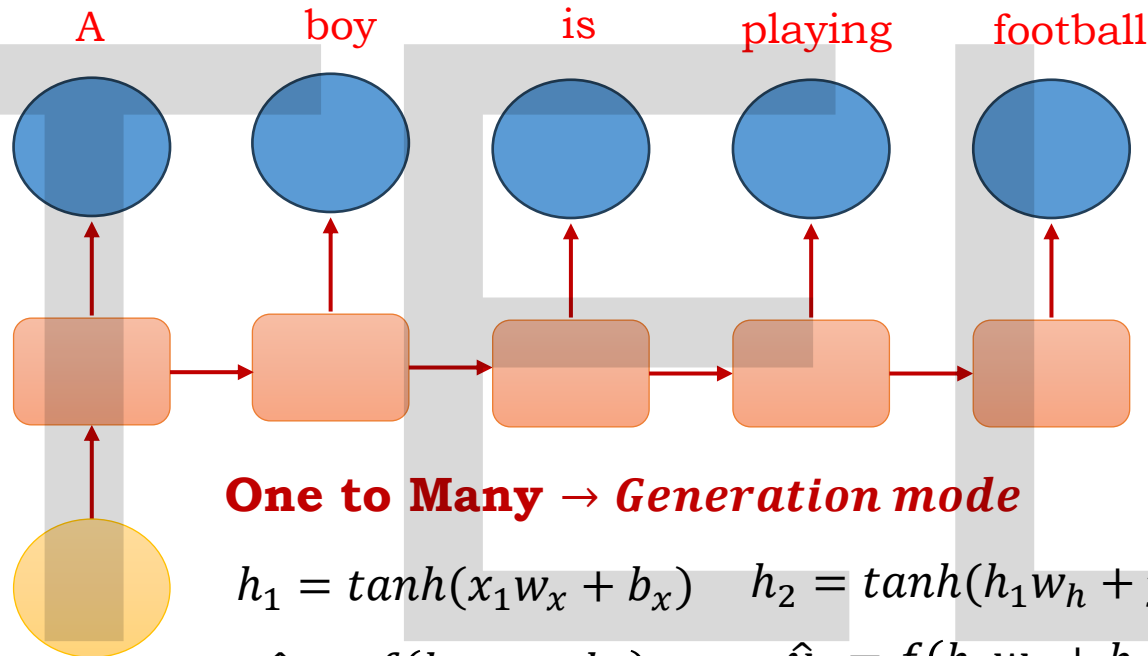
e.g. Image Captioning

Image -> sequence of words

e.g. Story generation

Single word -> sequence of words

Once upon a time, two friends... (story continues)



One to Many → Generation mode

$$h_1 = \tanh(x_1 w_x + b_x) \quad h_2 = \tanh(h_1 w_h + \hat{y}_1 w_y)$$

$$\hat{y}_1 = f(h_1 w_o + b_o) \quad \hat{y}_2 = f(h_2 w_o + b_o)$$

The process continues

So, we get a **sequence of outputs**, even though we started with **one input**.



"Friendship"

RNN: Types

e.g. Sentiment classification

Sequence of words -> sentiment

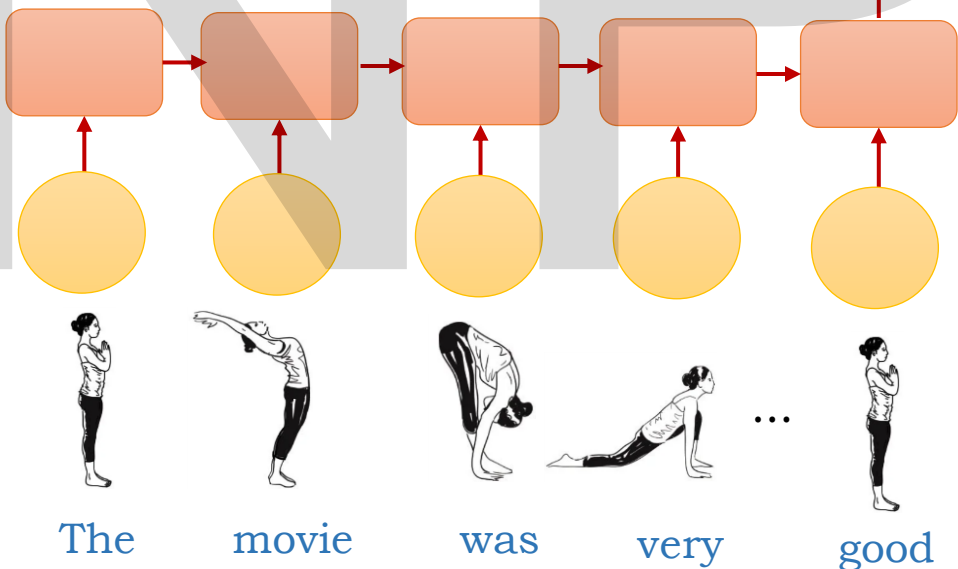
Positive

e.g. action prediction

Sequence of video frames -> action class

Surya
Namaskar

Many to One



Surya namaskar Example credit: IIT Madras CS7015 Deep Learning course (Lecture series 13.1)

e.g. Machine Translation

Sequence of words ->
sequence of words

e.g. Video Captioning

Sequence of video frames -> caption

e.g. POS tagging

Noun

Verb

Adverb

Determiner

Noun

Pranamasana

Hastauttanasana

...

...

...

आप

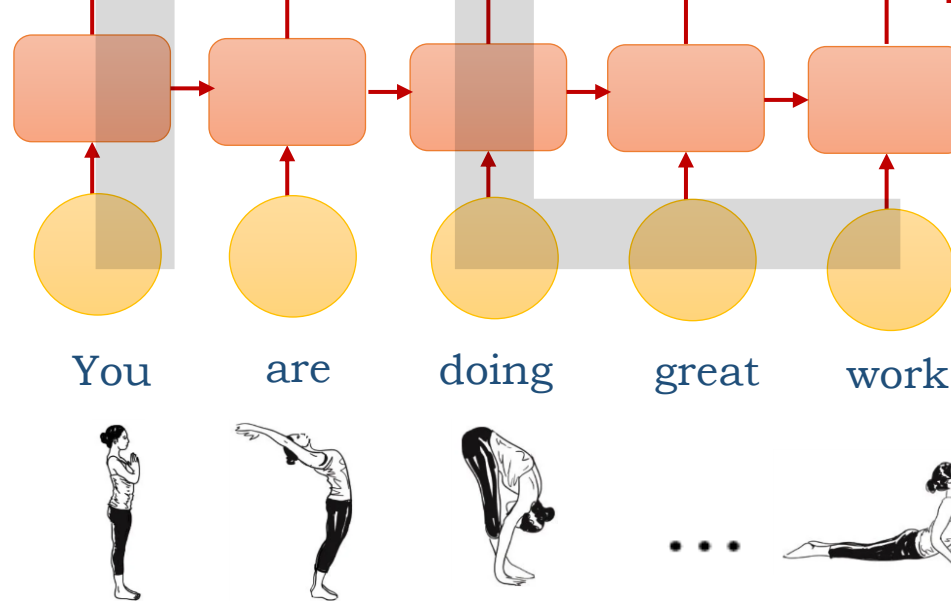
बहुत

अच्छा

काम

कर रहे हैं

Many to Many



Children

play

outside

every

evening

Sequence of
video frames

Summary

- Why FFNN/CNN fail for sequences
- What sequence learning problem needs
- RNN architecture
- Types of RNN

Next Session

Solving a numerical example on RNN (Input $\rightarrow$ Hidden $\rightarrow$ Output)

NLP TREL

NIPTELL

Thank you